

▼ Problem Statement

Yulu is India's leading micro-mobility service provider, which offers unique vehicles for the daily commute. Starting off as a mission to eliminate traffic congestion in India, Yulu provides the safest commute solution through a user-friendly mobile app to enable shared, solo and sustainable commuting.

Yulu zones are located at all the appropriate locations (including metro stations, bus stands, office spaces, residential areas, corporate offices, etc) to make those first and last-miles smooth, affordable, and convenient! Yulu has recently suffered considerable dips in their revenues. They have contracted a consulting company to understand the factors on which the demand for these shared electric cycles depends. Specifically, they want to understand the factors affecting the demand for these shared electric cycles in the American market.

How you can help here?

The company wants to know:

- Which variables are significant in predicting the demand for shared electric cycles in the American market?
- How well those variables describe the electric cycle demands

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from scipy.stats import ttest_ind # T-test for independent samples
from scipy.stats import shapiro # Shapiro-Wilk's test for Normality
from scipy.stats import levene # Levene's test for Equality of Variance
from scipy.stats import f_oneway # One-way ANOVA
from scipy.stats import kruskal
from scipy.stats import chi2_contingency # Chi-square test of independence
```

```
from google.colab import files
uploaded = files.upload()
```

[Choose Files](#) bike_sharing.csv
bike_sharing.csv(text/csv) - 648353 bytes, last modified: 5/21/2026 - 100% done
Saving bike_sharing.csv to bike_sharing.csv

```
df=pd.read_csv("bike_sharing.csv")
```

```
df.shape
```

```
(10886, 12)
```

```
df.head()
```

	datetime	season	holiday	workingday	weather	temp	atemp	humidity	windspeed	casual	registered	count
0	2011-01-01 00:00:00	1	0	0	1	9.84	14.395	81	0.0	3	13	16
1	2011-01-01 01:00:00	1	0	0	1	9.02	13.635	80	0.0	8	32	40
2	2011-01-01 02:00:00	1	0	0	1	9.02	13.635	80	0.0	5	27	32
3	2011-01-01 03:00:00	1	0	0	1	9.84	14.395	75	0.0	3	10	13
4	2011-01-01 04:00:00	1	0	0	1	9.84	14.395	75	0.0	0	1	1

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10886 entries, 0 to 10885
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   datetime    10886 non-null object
1   season      10886 non-null int64
2   holiday     10886 non-null int64
3   workingday  10886 non-null int64
4   weather     10886 non-null int64
5   temp        10886 non-null float64
6   atemp       10886 non-null float64
7   humidity    10886 non-null int64
8   windspeed   10886 non-null float64
9   casual      10886 non-null int64
10  registered  10886 non-null int64
11  count       10886 non-null int64
dtypes: float64(3), int64(8), object(1)
memory usage: 1020.7+ KB
```

```
df.describe()
```

	season	holiday	workingday	weather	temp	atemp	humidity	windspeed	casual	registered	count
count	10886.000000	10886.000000	10886.000000	10886.000000	10886.000000	10886.000000	10886.000000	10886.000000	10886.000000	10886.000000	10886.000000
mean	2.506614	0.028569	0.680875	1.418427	20.23086	23.655084	61.886460	12.799395	36.021955	155.552177	191.574132
std	1.116174	0.166599	0.466159	0.633839	7.79159	8.474601	19.245033	8.164537	49.960477	151.039033	181.144454
min	1.000000	0.000000	0.000000	1.000000	0.82000	0.760000	0.000000	0.000000	0.000000	0.000000	1.000000
25%	2.000000	0.000000	0.000000	1.000000	13.94000	16.665000	47.000000	7.001500	4.000000	36.000000	42.000000
50%	3.000000	0.000000	1.000000	1.000000	20.50000	24.240000	62.000000	12.998000	17.000000	118.000000	145.000000
75%	4.000000	0.000000	1.000000	2.000000	26.24000	31.060000	77.000000	16.997900	49.000000	222.000000	284.000000
max	4.000000	1.000000	1.000000	4.000000	41.00000	45.455000	100.000000	56.996900	367.000000	886.000000	977.000000

```
df.describe(include="O")
```

	datetime
count	10886
unique	10886
top	2012-12-19 23:00:00
freq	1

▼ Detail Analysis

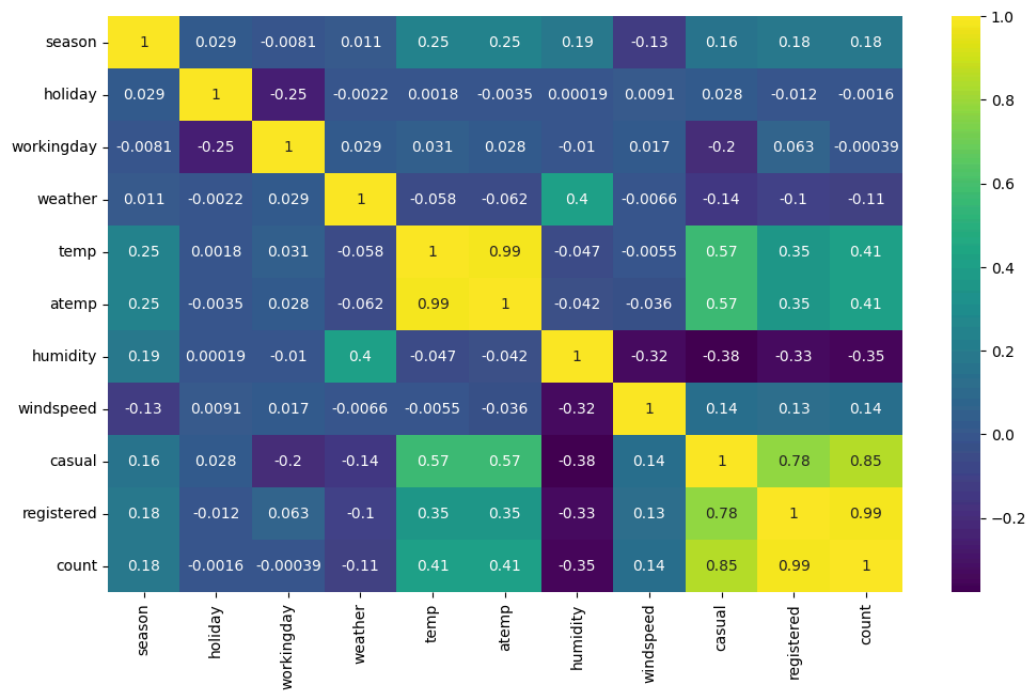
```
def dist_check(df,col_name):
    print("Unique values",df[col_name].unique())
    print("Value coounts", df[col_name].value_counts())
```

```
col_list=["season","holiday","workingday","weather"]
for items in col_list:
    print(f"Column Name: {items}")
    dist_check(df,col_name=items)
    print("-"*10)
```

```
Column Name: season
Unique values [1 2 3 4]
Value coounts season
4    2734
2    2733
3    2733
1    2686
Name: count, dtype: int64
-----
Column Name: holiday
Unique values [0 1]
Value coounts holiday
0    10575
1     311
Name: count, dtype: int64
-----
Column Name: workingday
Unique values [0 1]
Value coounts workingday
1    7412
0    3474
Name: count, dtype: int64
-----
Column Name: weather
Unique values [1 2 3 4]
Value coounts weather
1    7192
2    2834
3     859
4         1
Name: count, dtype: int64
-----
```

```
df_only_num=df.select_dtypes(include=["float","int"])
```

```
plt.figure(figsize=(12, 7)) ## This define my canvas size.
sns.heatmap(df_only_num.corr(method='spearman'),
            annot=True, cmap='viridis')
plt.show()
```



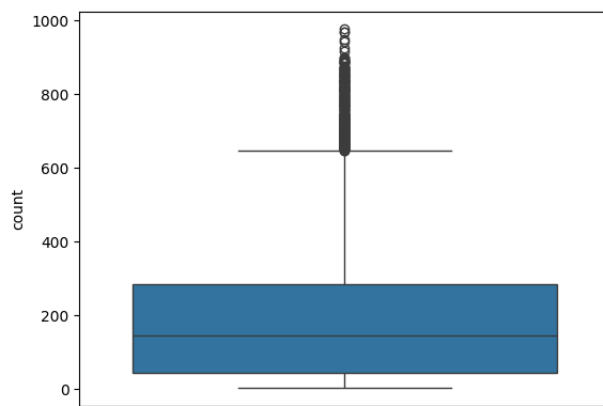
```
dfn=df.drop(columns=['casual','registered','atemp'])
```

```
dfn.head()
```

	datetime	season	holiday	workingday	weather	temp	humidity	windspeed	count
0	2011-01-01 00:00:00	1	0	0	1	9.84	81	0.0	16
1	2011-01-01 01:00:00	1	0	0	1	9.02	80	0.0	40
2	2011-01-01 02:00:00	1	0	0	1	9.02	80	0.0	32
3	2011-01-01 03:00:00	1	0	0	1	9.84	75	0.0	13
4	2011-01-01 04:00:00	1	0	0	1	9.84	75	0.0	1

Next steps: [Generate code with dfn](#) [New interactive sheet](#)

```
sns.boxplot(y=dfn['count'])
plt.show()
```



```
sns.distplot(np.log(dfn['count']),bins=10)
plt.show()
```

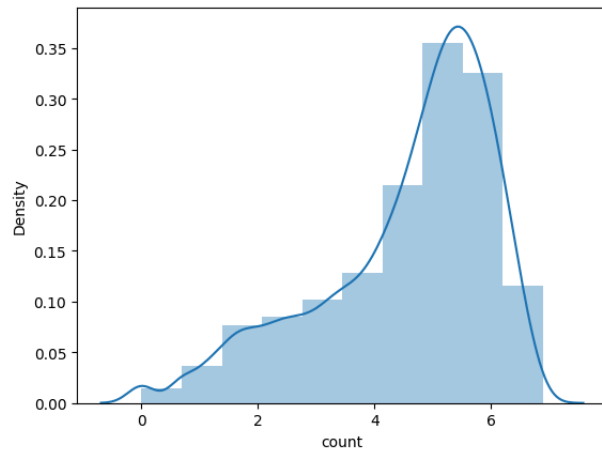
```
/tmp/ipykernel_1706/242116206.py:1: UserWarning:
```

```
`distplot` is a deprecated function and will be removed in seaborn v0.14.0.
```

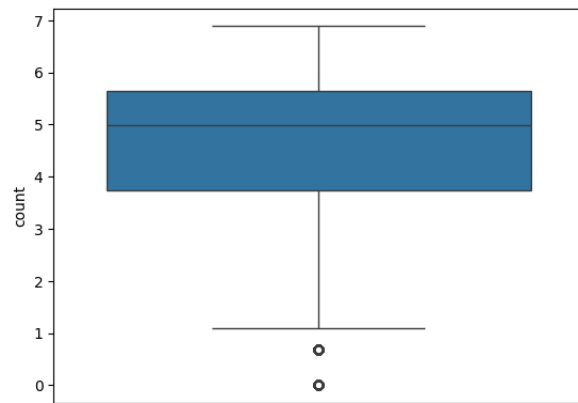
Please adapt your code to use either ``displot`` (a figure-level function with similar flexibility) or ``histplot`` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
sns.distplot(np.log(dfn['count']),bins=10)
```



```
sns.boxplot(y=np.log(dfn['count']))  
plt.show()
```



```
sns.distplot(np.log(dfn['count']),bins=3)
```

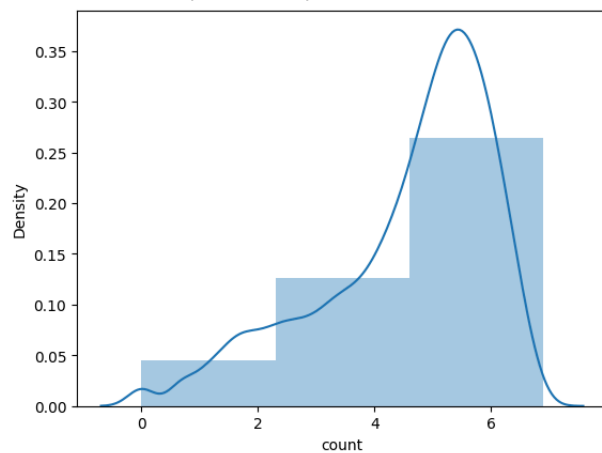
/tmp/ipykernel_1706/1267814950.py:1: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
sns.distplot(np.log(dfn['count']),bins=3)
<Axes: xlabel='count', ylabel='Density'>
```



✓ Ques. 1 - Is there any significant difference between the no. of bike rides on weekdays and weekends?

H_0 : The demand of bikes on weekdays is greater or similar to the demand of bikes on weekend.

H_a : The demand of bikes on weekdays is less than the demand of bikes on weekend.

Let μ_1 and μ_2 be the average no. of bikes rented on weekdays and weekends respectively.

Mathematically, the above formulated hypothesis can be written as:

$H_0 : \mu_1 \geq \mu_2$

$H_a : \mu_1 < \mu_2$

```
dfn.groupby('workingday')['count'].size()
```

	count
workingday	
0	3474
1	7412

dtype: int64

```
weekday=dfn[dfn['workingday']==1]['count'].sample(3000)
weekend=dfn[dfn['workingday']==0]['count'].sample(3000)
```

```
round(weekday.std(), 2)
```

185.29

```
round(weekend.std(),2)
```

174.53

alpha=0.05

Start coding or [generate](#) with AI.

```
def result(p_value, alpha):
    if p_value < alpha:
        print(f'As the p-value {p_value} is less than the level of significance, we reject the null hypothesis.')
    else:
        print(f'As the p-value {p_value} is greater than the level of significance, we fail to reject the null hypothesis.')
```

```
test_stat,p_value=ttest_ind(weekday,weekend,equal_var=False,alternative="less")
```

p_value

np.float64(0.7739505224158406)

```
result(p_value , alpha)
```

As the p-value 0.7739505224158406 is greater than the level of significance, we fail to reject the null hypothesis.

Is the demand of bicycles on rent same for different weather conditions?

```
dfn.groupby('weather')['count'].describe()
```

	count	mean	std	min	25%	50%	75%	max	
weather									
1	7192.0	205.236791	187.959566	1.0	48.0	161.0	305.0	977.0	
2	2834.0	178.955540	168.366413	1.0	41.0	134.0	264.0	890.0	
3	859.0	118.846333	138.581297	1.0	23.0	71.0	161.0	891.0	
4	1.0	164.000000	NaN	164.0	164.0	164.0	164.0	164.0	

✓ Step 1: Define the null and alternate hypothesis

H_0 : The average no. of bike rides in different weather conditions are equal.

H_a : The average no. of bike rides in different weather conditions are not equal.

Let μ_1 and μ_2 be the average no. of bikes rented on weekdays and weekends respectively.

```
dfn=dfn[~(dfn['weather']==4)]
```

```
w1=dfn[dfn['weather']==1]['count'].sample(750)
w2=dfn[dfn['weather']==2]['count'].sample(750)
w3=dfn[dfn['weather']==3]['count'].sample(750)
#w4=dfn[dfn['weather']==4]['count'].sample(750)
#
```

✓ Shapiro test

```
## Ho : Normal Ha: Npt normal
```

```
w,p_value=shapiro(dfn['count'].sample(5000))
result(p_value,alpha)
```

As the p-value 8.915626398186662e-53 is less than the level of significance, we reject the null hypothesis.

```
stat,p_value=kruskal(w1,w2,w3)
result(p_value,alpha)
```

As the p-value 1.0229519261648739e-20 is less than the level of significance, we reject the null hypothesis.

Are the weather conditions significantly different during different seasons?

Double-click (or enter) to edit

✓ Step 1: Define the null and alternate hypothesis

H_0 : Weather conditions are independent of the season.

H_a : Weather condition depends on the ongoing season.

```
dict1={1:'Sunny',
       2:'Cloudy',
       3:'Rainy'}
```

```
dfn["weather_enc"]=dfn["weather"].map(dict1)
```

```
dict2={1:'Summer',
       2:'Monsoon',
       3:'Winter',
       4:'Autumn'}
```

```
dfn["season_enc"]=dfn["season"].map(dict2)
```

```
contingency=pd.crosstab(dfn.season_enc,dfn.weather_enc)
```

```
contingency
```

weather_enc Cloudy Rainy Sunny 

season_enc 

Autumn	807	225	1702
Monsoon	708	224	1801
Summer	715	211	1759
Winter	604	199	1930

Next steps: [Generate code with contingency](#) [New interactive sheet](#)